PRUEBA TÉCNICA · 4 DÍAS CALENDARIO · GITHUB

# FleetPulse: Telemetría e Incident Copilot

Wilson Jacob Guerrero Chaves · Ingeniero Full Stack

> **Objetivo.** Construir una aplicación Full Stack capaz de recibir eventos GPS sintéticos en vivo y dar soporte a operadores con un módulo de recuperación documental segura (RAG). Se evalúa arquitectura, integridad, ingeniería con agentes de IA y capacidad para explicar decisiones verificables.

## Condiciones generales

Dispondrás de **4 días calendario (96 horas)** desde el correo formal; se indicará fecha y hora exacta en zona America/Bogota, con dedicación sugerida máxima de 16–20 horas efectivas. No uses datos reales de empleadores, servicios pagos obligatorios ni credenciales privadas. Priorizar P0 y documentar trade-offs.

## Base funcional común · 70 puntos

1. Backend Node.js + TypeScript, PostgreSQL con esquema/migraciones reproducibles.
2. API de ingesta, vehículos, historial, alertas, salud y SSE/WebSocket según docs/common-spec.md.
3. Validación parcial, idempotencia por eventId, eventos recibidos fuera de orden y alertas OVERSPEED cuando speedKph > 80.
4. Angular con tabla de estado de vehículos, búsqueda, historial y alertas; cambios visibles sin refrescar.
5. Reproductor GPS creado por el candidato: lee las 24 líneas del fixture en orden, intervalo configurable, iniciar, pausar, reanudar, reiniciar y reset de datos locales.
6. Primera reproducción con base limpia: 20 aceptados, 2 duplicados y 2 rechazados; 8 vehículos y 4 alertas. Segunda sin reset: 0 aceptados, 22 duplicados y 2 rechazados, sin alertas nuevas.
7. Protección de ingesta por API key en entorno, validación y manejo seguro de secretos.
8. Docker Compose (mínimo PostgreSQL), README de arranque y ejecución y GitHub Actions (lint/typecheck, pruebas).
9. Pruebas automatizadas: duplicados, inválidos, timestamps, umbral 80, seguridad y reconexión.

## Prueba reproducible de tiempo real

El reproductor debe enviar progresivamente los 24 eventos originales a la API y mostrar variaciones de estado y alertas en Angular **sin actualizar la página**. No basta cargar todos los datos en un solo lote. Una CLI sencilla es suficiente.

> **Primera ejecución, base limpia:** 24 entradas → 20 nuevas, 2 duplicadas, 2 rechazadas; 8 vehículos, 4 alertas.
> **Segunda ejecución, sin borrar datos:** 0 nuevas, 22 duplicadas, 2 rechazadas; 20 eventos persistidos y 4 alertas.

Contratos detallados en **docs/common-spec.md**, **docs/DEMO-REPLAY.md** y oráculos de **fixtures/**.

## Reto especializado · 30 puntos

Implementar **FleetPulse Incident Copilot**, un componente que responde consultas de soporte fundamentándose exclusivamente en runbooks sintéticos y datos/alertas registrados. Los archivos de **knowledge/** son fuentes no confiables para evaluar recuperación, citas, abstención y seguridad.

1. P0 · RE-01 (10 pts): construir recuperación real y explicable de documentos Markdown sintéticos de knowledge/, con fuente, fragmento y traza.
2. P0 · RE-02 (8 pts): POST /api/support/ask: respuestas fundamentadas con citas verificables y grounded; correlación con alertas de FleetPulse cuando proceda, abstención si no existe evidencia.

3. P0 · RE-03 (8 pts): tratar la inyección maliciosa en KB-99 como dato no confiable; proteger secretos, impedir efectos laterales y automatizar casos de seguridad.
4. P0 · RE-04 (4 pts): ejecutar casos de fixtures/support_questions.json, entregar evaluación y explicar arquitectura, uso de agentes y decisiones humanas.
5. P1 opcional, sin penalización: modelo LLM, embeddings/pgvector, UI Angular de chat, despliegue cloud, procesamiento de audio o mensajería avanzada.

El producto debe funcionar y poder evaluarse **sin licencias ni API comerciales**: se admite recuperación con PostgreSQL full-text o búsqueda lexical y respuesta extractiva sustentada, además de LLM local/gratuito opcional. El núcleo de ingeniería agéntica para codificar sigue siendo obligatorio.

**Pruebas adversariales:** podrán formularse consultas parafraseadas, añadirse runbooks sintéticos y validarse citas de datos GPS (EVENT:id). El contrato de trazas, abstención y seguridad se detalla en docs/RETO-ESPECIALIZADO-RAG.md.

## Archivos y evidencia a entregar

1. Usar IA agéntica para DESARROLLAR (Codex, Claude Code, OpenCode u otra), con al menos dos iteraciones verificadas. No basta usar un chatbot; documentar verificaciones humanas.
2. Entregar GitHub público o privado con acceso a evaluadores, historial auténtico de commits y tag entrega-final.
3. Incluir README, documentación técnica y de IA en docs/AI-WORKFLOW.md, docs/RAG-SECURITY.md, docs/RAG-EVALUATION.md y docs/RETROSPECTIVA.md.
4. Pruebas automatizadas y CI funcionando; herramientas gratuitas. No incluir credenciales ni datos privados.
5. Video 8–12 minutos (máximo 15) enlazado en README: funcionamiento de Angular + replay, dos pasadas, consultas RAG con fuentes, rechazo seguro de instrucciones hostiles y decisiones de desarrollo.

## Criterios de evaluación

Base común 70 puntos: API/datos 25, Angular/tiempo real 15, calidad/seguridad 15, IA agéntica y criterio 10, entrega reproducible y video 5. Reto específico 30 puntos: recuperación 10, fundamentación 8, seguridad 8, decisiones y evidencias 4.

## Demostración y prioridades

Mostrar en video: arranque desde datos limpios, reproducción gradual con pausa/reanudación, estado actualizado por SSE/WS, segundo replay idempotente, consulta OVERSPEED con citas y traza, pregunta no respaldada y prueba ante KB-99. Comentar decisiones humanas y un error corregido a la IA.

**P0:** base funcional probada y módulo especializado mínimo. **P1:** mapa, chat web, embeddings avanzados, LLM externo y despliegue cloud. No se penaliza omitir P1.

El kit incluye documentación y fixtures oficiales. Prevalecen docs/common-spec.md para comportamiento GPS, docs/DEMO-REPLAY.md para simulación y docs/RETO-ESPECIALIZADO-RAG.md para el reto específico.